

Document number: p0544R0

Date: 2017-02-01

Audience: Library Evolution Working Group

Reply to: Titus Winters <titus@google.com>, Geoff Romer <gromer@google.com>

User Injection of Filesystems

1. [Rationale](#)
2. [Design Choices](#)
3. [Proposed Wording](#)

Rationale

The existing proposals in the Filesystem TS (now adopted for inclusion with C++17) effectively specify the C++ filesystem API as a set of C++ wrappers and types (like `directory_iterator`) directly on top of whatever is provided by the OS for filesystem interaction. This is easy to specify and implement, but inflexible and likely does not provide the functionality needed for a modern C++ codebase — for example, it is impossible to fake filesystem errors in testing, or to use the same interfaces for in-memory filesystems (avoiding the essential OS and I/O overhead). Our belief, based on extensive experience in the Google codebase and production systems, is that most robust C++ installations will wind up wrapping the proposed interface in order to provide opportunities to change the backing store, simulate errors for testing, etc. To that end, we propose the definition of a filesystem type, and injection mechanisms to allow user-customization of the filesystem backend used by the free functions provided by the Filesystem TS.

This proposal is pure-library, OS and compiler agnostic, and is backwards compatible with any existing code written against the Filesystem TS. It has not yet been implemented in this form. However, the filesystems at use in Google production work in roughly this way with small differences. In the Google design, multiple filesystems are present at once, they are all pinned into the filesystem at some root (`/memfile`, `/virtualfs`, etc) with the OS local filesystem as the generic fallback. This works for us because we have no symlinks. Once symlinks are in the equation, the design has to shift to something like what we are describing.

Design Choices

The biggest point of discussion in the design for filesystem injection is how to specify where injection takes place. Is it a new filesystem root? If so, how is it specified (especially on POSIX systems)? If you can inject at any point in a filesystem, does that injection respect symlinks? If so, that requires one (or more) OS calls on every filesystem call to resolve symlinks in order to determine if the specified path(s) fall inside the injection point. Since we cannot determine one policy that will satisfy all users for the above questions, we instead choose to present a more general injection mechanism: replace the filesystem entirely. In conjunction with the ability to get the “default” filesystem, a user can then build a solution that forwards to the underlying filesystem in a fashion congruent with any of the above designs.

Under this design, the expected cost is bounded by following an atomic pointer plus a vtable dereference. In comparison to the overhead of performing I/O through the OS, this is expected to be negligible.

Proposed Wording

Changes are relative to [N4582](#).

Add the following to the `<filesystem>` synopsis in `[fs.filesystem.syn]`:

```
...

class filesystem_error;
class directory_entry;

class filesystem;
class default_filesystem_impl; // exposition only
class directory_traverser;

class directory_iterator;

// range access for directory_iterator
directory_iterator begin(directory_iterator iter) noexcept;
directory_iterator end(const directory_iterator&) noexcept;
```

```

bool operator==(const directory_iterator& lhs, const directory_iterator& rhs);
bool operator!=(const directory_iterator& lhs, const directory_iterator& rhs);

class recursive_directory_iterator;

...

typedef chrono::time_point<trivial-clock> file_time_type;

// filesystem injection
extern atomic<filesystem*> current filesystem; // exposition only
void inject filesystem(filesystem& fs);
filesystem& default filesystem();

// filesystem operation functions

...

```

Add a new paragraph to the end of [fs.err.report] as follows:

When a function that does not take an `error_code` parameter is specified to call a function that takes an `error_code` parameter, the outer function is understood to construct a local error code variable `ec` to pass to the function call. If `!ec` after the call completes, unless otherwise specified the function throws a `filesystem_error` exception containing `ec`, an implementation-defined message, and any paths that were passed to the function.

Insert new sections above [class.directory_iterator]:

27.10.? Class filesystem [class.filesystem]

Class `filesystem` defines the base class for the types of objects that provide access to the fundamental operations of a specific file system. A special `filesystem` object (`[class.default filesystem impl]`) provides access to the native file system provided by the operating system, and other `filesystem` objects can implement virtual file systems for purposes such as testing.

Classes derived from `filesystem` shall satisfy all of the requirements of `filesystem`.

```

class filesystem {
public:
    filesystem() = default;
    virtual ~filesystem();
    filesystem(const filesystem&) = delete;
    filesystem& operator=(const filesystem&) = delete;

    virtual bool copy_file(const path& from, const path& to,
                          copy_options options, error_code& ec) noexcept = 0;
    virtual bool create_directory(const path& p, error_code& ec) noexcept = 0;
    virtual bool create_directory(const path& p, const path& existing_p,
                                error_code& ec) noexcept = 0;
    virtual void create_directory_symlink(
        const path& to, const path& new_symlink, error_code& ec) noexcept = 0;
    virtual void create_hard_link(const path& to, const path& new_hard_link,
                                error_code& ec) noexcept = 0;
    virtual void create_symlink(const path& to, const path& new_symlink,
                                error_code& ec) noexcept = 0;
    virtual path current_path(error_code& ec) const = 0;
    virtual void current_path(const path& p, error_code& ec) noexcept = 0;
    virtual bool equivalent(const path& p1, const path& p2, error_code& ec) const noexcept = 0;
    virtual uintmax_t file_size(const path& p, error_code& ec) const noexcept = 0;
    virtual uintmax_t hard_link_count(const path& p, error_code& ec) const noexcept = 0;
    virtual file_time_type last_write_time(const path& p, error_code& ec) const noexcept = 0;
    virtual void last_write_time(const path& p, file_time_type new_time,
                                error_code& ec) const noexcept = 0;
    virtual void permissions(const path& p, perms prms, error_code& ec) const noexcept = 0;
    virtual path read_symlink(const path& p, error_code& ec) const = 0;
    virtual bool remove(const path& p, error_code& ec) noexcept = 0;
    virtual void rename(const path& old_p, const path& new_p, error_code& ec) noexcept = 0;
    virtual void resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept = 0;
    virtual space_info space(const path& p, error_code& ec) const noexcept = 0;
    virtual file_status status(const path& p, error_code& ec) const noexcept = 0;
    virtual file_status symlink_status(const path& p, error_code& ec) const noexcept = 0;
    virtual path system_complete(const path& p, error_code& ec) const = 0;

```

```
virtual path temp_directory_path(error_code& ec) = 0;

virtual unique_ptr<directory_traverser> directory_traverser(
    const path& p, directory_options options, error_code& ec) const = 0;
};
```

27.10.?.1 filesystem members [filesystem.members]

27.10.?.1.1 Copy file [filesystem.copy_file]

```
virtual bool copy_file(const path& from, const path& to,
    copy_options options, error_code& ec) noexcept = 0;
```

Copy [fs.op.copy_file] paragraphs 3 to 9 here, with edits as specified:

Requires: At most one constant from each copy_options group is present in options.

Effects: Report a file already exists error as specified in Error reporting (27.5.6.5) if:

- exists(to) and equivalent(from, to), OR
- exists(to) and (options & (copy_options::skip_existing | copy_options::overwrite_existing | copy_options::update_existing)) == copy_options::none.

Otherwise copy the contents and attributes of the file from resolves to to the file to resolves to if:

- !exists(to), OR
- exists(to) and (options & copy_options::overwrite_existing) != copy_options::none, OR
- exists(to) and (options & copy_options::update_existing) != copy_options::none, and from is more recent than to, determined as if by use of the last_write_time function.

Otherwise no effects.

Returns: true if the from file was copied, otherwise false. ~~The signature with argument ec~~ **r**Return false if an error occurs.

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

Complexity: At most one direct or indirect invocation of status(to).

27.10.?.1.2 Create directory [filesystem.create_directory]

```
virtual bool create_directory(const path& p, error_code& ec) noexcept = 0;
```

Copy [fs.op.create_directory] paragraphs 1 to 4 here, with edits as specified:

Effects: Establishes the postcondition by attempting to create the directory p resolves to, ~~as if by POSIX mkdir() with a second argument of static_cast<int>(perms::all).~~ Creation failure because p resolves to an existing directory shall not be treated as an error.

Postcondition: is_directory(p).

Returns: true if a new directory was created, otherwise false. ~~The signature with argument ec~~ **r**Returns false if an error occurs.

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

```
virtual bool create_directory(const path& p, const path& existing_p,
    error_code& ec) noexcept = 0;
```

Copy [fs.op.create_directory] paragraphs 5 to 8 here, with edits as specified:

Effects: Establishes the postcondition by attempting to create the directory p resolves to, with attributes copied from directory existing_p. The set of attributes copied is operating system dependent. Creation failure because p resolves to an existing directory shall not be treated as an error. [Note: For POSIX based operating systems the attributes are those copied by native API stat(existing_p.c_str(), &attributes_stat) followed by mkdir(p.c_str(), attributes_stat.st_mode). For Windows based operating systems the attributes are those copied by native API CreateDirectoryExW(existing_p.c_str(), p.c_str(), 0). — end note]

Postcondition: is_directory(p).

Returns: true if a new directory was created, otherwise false. ~~The signature with argument ec~~ **r**Returns false

if an error occurs.

Throws: As specified in Error reporting (27.5.6.5):

27.10.?.1.3 Create directory symlink [filesystem.create_directory_symlink]

```
virtual void create_directory_symlink(  
    const path& to, const path& new_symlink, error_code& ec) noexcept = 0;
```

Copy [fs.op.create_dir_symlk] paragraphs 1 to 5 here, with edits as specified:

Effects: Establishes the postcondition, ~~as if by POSIX symlink()~~.

Postcondition: new_symlink resolves to a symbolic link file that contains an unspecified representation of to.

Throws: As specified in Error reporting (27.5.6.5):

~~[Note: Some operating systems require symlink creation to identify that the link is to a directory. Portable code should use create_directory_symlink() to create directory symlinks rather than create_symlink() — end note]~~

~~[Note: Some operating systems do not support symbolic links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support symbolic links regardless of the operating system. — end note]~~

27.10.?.1.4 Create hard link [filesystem.create_hard_link]

```
virtual void create_hard_link(const path& to, const path& new_hard_link,  
    error_code& ec) noexcept = 0;
```

Copy [fs.op.create_hard_lk] paragraphs 1 to 4 here, with edits as specified:

Effects: Establishes the postcondition, ~~as if by POSIX link()~~.

Postcondition:

- exists(to) && exists(new_hard_link) && equivalent(to, new_hard_link)
- The contents of the file or directory to resolves to are unchanged.

Throws: As specified in Error reporting (27.5.6.5):

~~[Note: Some operating systems do not support hard links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support hard links regardless of the operating system. Some file systems limit the number of links per file. — end note]~~

27.10.?.1.5 Create symlink [filesystem.create_symlink]

```
virtual void create_symlink(const path& to, const path& new_symlink,  
    error_code& ec) noexcept = 0;
```

Copy [fs.op.create_symlink] paragraphs 1 to 4 here, with edits as specified:

Effects: Establishes the postcondition, ~~as if by POSIX symlink()~~.

Postcondition: new_symlink resolves to a symbolic link file that contains an unspecified representation of to.

Throws: As specified in Error reporting (27.5.6.5):

~~[Note: Some operating systems do not support symbolic links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support symbolic links regardless of the operating system. — end note]~~

27.10.?.1.6 Current path [filesystem.current_path]

```
virtual path current_path(error_code& ec) const = 0;
```

Copy [fs.op.current_path] paragraphs 1 to 5 here, with edits as specified:

Returns: The absolute path of the current working directory, ~~obtained as if by POSIX `getcwd()`. The signature with argument `ec`~~`r` Returns `path()` if an error occurs.

Throws: As specified in Error reporting (27.5.6.5):

Remarks: The current working directory is the directory, associated with the process, that is used as the starting location in pathname resolution for relative paths.

[Note: The `current_path()` name was chosen to emphasize that the return is a path, not just a single directory name.

The current path as returned by many operating systems is a dangerous global variable. It may be changed unexpectedly by a third-party or system library functions, or by another thread. — end note

```
virtual void current_path(const path& p, error_code& ec) noexcept = 0;
```

Copy [fs.op.current_path] paragraphs 6 to 9 here, with edits as specified:

Effects: Establishes the postcondition, ~~as if by POSIX `chdir()`.~~

Postcondition: `equivalent(p, current_path())`.

Throws: As specified in Error reporting (27.5.6.5):

[Note: The current path for many operating systems is a dangerous global state. It may be changed unexpectedly by a third-party or system library functions, or by another thread. — end note]

27.10.2.1.7 Equivalent [filesystem.equivalent]

```
virtual bool equivalent(const path& p1, const path& p2, error_code& ec) const noexcept = 0
```

Copy [fs.op.equivalent] paragraphs 1 to 4 here, with edits as specified:

Effects: Determines `file_status s1` and `s2`, as if by `status(p1)` and `status(p2)`, respectively.

Returns: `true`, if `s1 == s2` and `p1` and `p2` resolve to the same file system entity, else `false`. ~~The signature with argument `ec`~~`r` Returns `false` if an error occurs.

Two paths are considered to resolve to the same file system entity if two candidate entities reside on the same device at the same location. ~~This is determined as if by the values of the POSIX `stat` structure, obtained as if by `stat()` for the two paths, having equal `st_dev` values and equal `st_ino` values.~~

Throws: `filesystem_error` if `(!exists(s2)) || (is_other(s1) && is_other(s2))`; otherwise as specified in Error reporting (27.5.6.5):

27.10.2.1.8 File size [filesystem.file_size]

```
virtual uintmax_t file_size(const path& p, error_code& ec) const noexcept = 0;
```

Copy [fs.op.file_size] paragraphs 1 to 2 here, with edits as specified:

Returns: If `!exists(p) || !is_regular_file(p)` an error is reported 27.5.6.5. Otherwise, the size in bytes of the file `p` resolves to, ~~determined as if by the value of the POSIX `stat` structure member `st_size` obtained as if by POSIX `stat()`. The signature with argument `ec`~~`r` Returns `static_cast<uintmax_t>(-1)` if an error occurs.

Throws: As specified in Error reporting (27.5.6.5):

27.10.2.1.9 Hard link count [filesystem.hard_link_count]

```
virtual uintmax_t hard_link_count(const path& p, error_code& ec) const noexcept = 0;
```

Copy [fs.op.hard_lk_ct] paragraphs 1 to 2 here, with edits as specified:

Returns: The number of hard links for `p`. ~~The signature with argument `ec` returns `static_cast<uintmax_t>` (-1) if an error occurs.~~

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

27.10.2.1.10 Last write time [filesystem.last write time]

```
virtual file_time_type last_write_time(const path& p, error_code& ec) const noexcept = 0;
```

Copy [fs.op.last_write_time] paragraphs 1 to 2 here, with edits as specified:

Returns: The time of last data modification of `p`, ~~determined as if by the value of the POSIX `stat` structure member `st_mtime` obtained as if by POSIX `stat()`.~~ ~~The signature with argument `ec` returns `file_time_type::min()` if an error occurs.~~

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

```
virtual void last_write_time(const path& p, file_time_type new_time,  
                             error_code& ec) const noexcept = 0;
```

Copy [fs.op.last_write_time] paragraphs 3 to 5 here, with edits as specified:

Effects: Sets the time of last modification of the file resolved to by `p` to `new_time`, ~~as if by POSIX `futimens()`.~~

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

~~[Note: A postcondition of `last_write_time(p) == new_time` is not specified since it might not hold for file systems with coarse time granularity. — end note]~~

27.10.2.1.11 Permissions [filesystem.permissions]

```
virtual void permissions(const path& p, perms prms, error_code& ec) const noexcept = 0;
```

Copy [fs.op.permissions] paragraphs 1 to 4 here, with edits as specified:

Requires: `!((prms & perms::add_perms) != perms::none <br> && (prms & perms::remove_perms) != perms::none)`.

Effects: Applies the effective permissions bits from `prms` to the file `p` resolves to, ~~as if by POSIX `fchmodat()`.~~ The effective permission bits are determined as specified in Table 150.

Table 150 — Effects of permission bits

Bits present in <code>prms</code>	Effective bits applied
Neither <code>add_perms</code> nor <code>remove_perms</code>	<code>prms & perms::mask</code>
<code>add_perms</code>	<code>status(p).permissions() (prms & perms::mask)</code>
<code>remove_perms</code>	<code>status(p).permissions() & (prms & perms::mask)</code>

~~[Note: Conceptually permissions are viewed as bits, but the actual implementation may use some other mechanism. — end note]~~

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

27.10.2.1.12 Read symlink [filesystem.read symlink]

```
virtual path read_symlink(const path& p, error_code& ec) const noexcept = 0;
```

Copy [fs.op.read_symlink] paragraphs 1 to 2 here, with edits as specified:

Returns: If `p` resolves to a symbolic link, a path object containing the contents of that symbolic link. ~~The signature with argument `ec` returns `path()` if an error occurs.~~

~~*Throws:* As specified in Error reporting (27.5.6.5).~~ [Note: It is an error if `p` does not resolve to a symbolic link.

— end note]

27.10.2.1.13 Remove [filesystem.remove]

```
virtual bool remove(const path& p, error_code& ec) noexcept = 0;
```

Copy [fs.op.remove] paragraphs 1 to 5 here, with edits as specified:

Effects: If `exists(symlink_status(p,ec))`, it is removed ~~as if by POSIX `remove()`.~~

[*Note:* A symbolic link is itself removed, rather than the file it resolves to being removed. — end note]

Postcondition: `!exists(symlink_status(p))`.

Returns: `false` if `p` did not exist, otherwise `true`. ~~The signature with argument `ec` returns `false` if an error occurs.~~

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

27.10.2.1.14 Rename [filesystem.rename]

```
virtual void rename(const path& old_p, const path& new_p, error_code& ec) noexcept = 0;
```

Copy [fs.op.rename] paragraphs 1 to 3 here, with edits as specified:

Effects: Renames `old_p` to `new_p`, ~~as if by POSIX `rename()`.~~

[*Note:* If `old_p` and `new_p` resolve to the same existing file, no action is taken. Otherwise, if `new_p` resolves to an existing non-directory file, it is removed, while if `new_p` resolves to an existing directory, it is removed if empty on POSIX compliant operating systems but is an error on some other operating systems. A symbolic link is itself renamed, rather than the file it resolves to being renamed. — end note]

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

27.10.2.1.15 Resize file [filesystem.resize_file]

```
virtual void resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept = 0;
```

Copy [fs.op.resize_file] paragraphs 1 to 3 here, with edits as specified:

Postcondition: `file_size() == new_size`.

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

~~*Remarks:* Achieves its postconditions as if by POSIX `truncate()`.~~

27.10.2.1.16 Space [filesystem.space]

```
virtual space_info space(const path& p, error_code& ec) const noexcept = 0;
```

Copy [fs.op.space] paragraphs 1 to 3 here, with edits as specified:

Returns: An object of type `space_info`. ~~The value of the `space_info` object is determined as if by using POSIX `statvfs` to obtain a POSIX `struct statvfs`, and then multiplying its `f_blocks`, `f_bfree`, and `f_bavail` members by its `f_frsize` member, and assigning the results to the `capacity`, `free`, and `available` members respectively. The object's members will be set as follows:~~

- capacity will be set to the total storage capacity of the storage volume containing `p`.
- free will be set to the total amount of free space on the storage volume containing `p`.
- available will be set to the amount of free space that is available to hold user data on the storage volume containing `p` [*Note:* This can be less than `free` because it excludes storage reserved for file system internals. — end note].

Any members for which the value cannot be determined shall be set to `static_cast<uintmax_t>(-1)`. ~~For the signature with argument `ec`, a~~All members are set to `static_cast<uintmax_t>(-1)` if an error occurs.

~~Throws: As specified in Error reporting (27.5.6.5).~~

~~Remarks: The value of member `space_info::available` is operating system dependent. [Note: `available` may be less than `free`. — end note]~~

27.10.?.1.17 Status [`filesystem.status`]

```
virtual file_status status(const path& p, error_code& ec) const noexcept = 0;
```

Copy [`fs.op.status`] paragraphs 4 to 9 here, with edits as specified:

Effects: If possible, determines the attributes of the file `p` resolves to, ~~as if by `POSIX::stat()`~~. If, during attribute determination, the underlying file system API reports an error, sets `ec` to indicate the specific error reported. Otherwise, `ec.clear()`.

[Note: This allows users to inspect the specifics of underlying API errors even when the value returned by `status()` is not `file_status(file_type::none)`. — end note]

Returns: If `ec != error_code()`:

- If the specific error indicates that `p` cannot be resolved because some element of the path does not exist, return `file_status(file_type::not_found)`.
- Otherwise, if the specific error indicates that `p` can be resolved but the attributes cannot be determined, return `file_status(file_type::unknown)`.
- Otherwise, return `file_status(file_type::none)`.

[Note: These semantics distinguish between `p` being known not to exist, `p` existing but not being able to determine its attributes, and there being an error that prevents even knowing if `p` exists. These distinctions are important to some use cases. — end note]

Otherwise,

- If the attributes indicate a regular file, ~~as if by `POSIX::S_ISREG`~~, return `file_status(file_type::regular)`. [Note: `file_type::regular` implies appropriate `<fstream>` operations would succeed, assuming no hardware, permission, access, or file system race errors. Lack of `file_type::regular` does not necessarily imply `<fstream>` operations would fail on a directory. — end note]
- Otherwise, if the attributes indicate a directory, ~~as if by `POSIX::S_ISDIR`~~, return `file_status(file_type::directory)`. [Note: `file_type::directory` implies `directory_iterator(p)` would succeed. — end note]
- Otherwise, if the attributes indicate a block special file, ~~as if by `POSIX::S_ISBLK`~~, return `file_status(file_type::block)`.
- Otherwise, if the attributes indicate a character special file, ~~as if by `POSIX::S_ISCHR`~~, return `file_status(file_type::character)`.
- Otherwise, if the attributes indicate a fifo or pipe file, ~~as if by `POSIX::S_ISFIFO`~~, return `file_status(file_type::fifo)`.
- Otherwise, if the attributes indicate a socket, ~~as if by `POSIX::S_ISSOCK`~~, return `file_status(file_type::socket)`.
- Otherwise, return `file_status(file_type::unknown)`.

Remarks: If a symbolic link is encountered during pathname resolution, pathname resolution continues using the contents of the symbolic link.

27.10.?.1.18 Symlink status [`filesystem.symlink_status`]

```
virtual file_status symlink_status(const path& p, error_code& ec) const noexcept = 0;
```

Copy [`fs.op.symlink_status`] paragraphs 1 to 4 here, with edits as specified:

Effects: Same as `status()`, above, except that the attributes of `p` are ~~determined as if by `POSIX::stat()`~~ always determined, even if `p` names a symbolic link.

Returns: Same as `status()`, above, except that if the attributes indicate a symbolic link, ~~as if by `POSIX_S_ISLNK`~~; return `file_status(file_type::symlink)`. ~~The signature with argument `ec`~~Returns `file_status(file_type::none)` if an error occurs.

Remarks: Pathname resolution terminates if `p` names a symbolic link.

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

27.10.2.1.19 System complete [filesystem.system_complete]

```
virtual path system_complete(const path& p, error_code& ec) const = 0;
```

Copy [`fs.op.system_complete`] paragraphs 1 to 6 here, with edits as specified:

Effects: Composes an absolute path from `p`, using the same rules used ~~by the operating system~~ to resolve a path passed ~~as the filename argument to standard library open functions~~ to other functions in this section.

Returns: The composed path. ~~The signature with argument `ec`~~Returns `path()` if an error occurs.

Postcondition: For the returned path, `rp`, `rp.is_absolute()` is true.

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

[*Example:* For POSIX based operating systems, `system_complete(p)` has the same semantics as `absolute(p, current_path())`.

For Windows based operating systems, `system_complete(p)` has the same semantics as `absolute(p, current_path())` if `p.is_absolute()` || `!p.has_root_name()` or `p` and `base` have the same `root_name()`. Otherwise it acts like `absolute(p, cwd)` is the current directory for the `p.root_name()` drive. This will be the current directory for that drive the last time it was set, and thus may be residue left over from a prior program run by the command processor. Although these semantics are useful, they may be surprising. — *end example*]

27.10.2.1.20 Temp directory path [filesystem.temp_directory_path]

```
virtual path temp_directory_path(error_code& ec) const = 0;
```

Copy [`fs.op.temp_dir_path`] paragraphs 1 to 4 here, with edits as specified:

Returns: An unspecified directory path suitable for temporary files. An error shall be reported if `!exists(p)` || `!is_directory(p)`, where `p` is the path to be returned. ~~The signature with argument `ec`~~Returns `path()` if an error occurs.

~~*Throws:* As specified in Error reporting (27.5.6.5).~~

[*Example:* For POSIX based operating systems, an implementation might return the path supplied by the first environment variable found in the list `TMPDIR`, `TMP`, `TEMP`, `TEMPDIR`, or if none of these are found, `"/tmp"`.

For Windows based operating systems, an implementation might return the path reported by the Windows `GetTempPath` API function. — *end example*]

27.10.2.1.21 Directory traverser [filesystem.directory_traverser]

```
virtual unique_ptr<directory_traverser> directory_traverser(  
    const path& p, directory_options options, error_code& ec) const = 0;
```

Copy [`directory_iterator.members`] paragraphs 2 to 4 here, with edits as specified:

Effects: For the directory that `p` resolves to, ~~constructs an iterator~~allocates a `directory_traverser` for the first element in a sequence of `directory_entry` elements representing the files in the directory, if any; otherwise the ~~end iterator~~past-the-end value. However, if

```
(options & directory_options::skip_permissions_denied) != directory_options::none
```

and construction encounters an error indicating that permission to access `p` is denied, constructs the ~~end~~

iterator past-the-end value and does not report an error.

Returns: a pointer to the allocated directory_traverser.

Throws: As specified in Error reporting (27.5.6.5).

[Note: To iterate over the current directory, use directory_iterator(".") rather than directory_iterator("").
—end note]

27.10.? Class default_filesystem_impl [class.default_filesystem_impl]

Class default_filesystem_impl is for exposition only. An implementation is permitted to provide equivalent functionality without providing a class with this name. This class provides a default implementation of the filesystem interface, which is implemented by delegating to the underlying operating system.

Descriptions are provided only where this class differs from the base class filesystem.

```
class default_filesystem_impl : public filesystem { // exposition only
public:
    virtual bool copy_file(const path& from, const path& to,
                           copy_options options, error_code& ec) noexcept;
    virtual bool create_directory(const path& p, error_code& ec) noexcept;
    virtual bool create_directory(const path& p, const path& existing_p,
                                   error_code& ec) noexcept;
    virtual void create_directory_symlink(
        const path& to, const path& new_symlink, error_code& ec) noexcept;
    virtual void create_hard_link(const path& to, const path& new_hard_link,
                                   error_code& ec) noexcept;
    virtual void create_symlink(const path& to, const path& new_symlink,
                                   error_code& ec) noexcept;
    virtual path current_path(error_code& ec);
    virtual void current_path(const path& p, error_code& ec) noexcept;
    virtual bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;
    virtual uintmax_t file_size(const path& p, error_code& ec) noexcept;
    virtual uintmax_t hard_link_count(const path& p, error_code& ec) noexcept;
    virtual file_time_type last_write_time(const path& p, error_code& ec) noexcept;
    virtual void last_write_time(const path& p, file_time_type new_time,
                                   error_code& ec) noexcept;
    virtual void permissions(const path& p, perms prms, error_code& ec) noexcept;
    virtual path read_symlink(const path& p, error_code& ec);
    virtual bool remove(const path& p, error_code& ec) noexcept;
    virtual void rename(const path& old_p, const path& new_p, error_code& ec) noexcept;
    virtual void resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept;
    virtual space_info space(const path& p, error_code& ec) noexcept;
    virtual file_status status(const path& p, error_code& ec) noexcept;
    virtual file_status symlink_status(const path& p, error_code& ec) noexcept;
    virtual path system_complete(const path& p, error_code& ec);
    virtual path temp_directory_path(error_code& ec);

    virtual unique_ptr<directory_traverser> directory_traverser(
        const path& p, directory_options options, error_code& ec);
};
```

27.10.?1 filesystem members [default_filesystem_impl.members]

27.10.?1.1 Create directory [default_filesystem_impl.create_directory]

virtual bool create_directory(const path& p, error_code& ec) noexcept;

Remarks: Achieves its postconditions as if by POSIX mkdir() with a second argument of static_cast<int>(perms::all).

27.10.?1.2 Create directory symlink [default_filesystem_impl.create_directory_symlink]

virtual void create_directory_symlink(
const path& to, const path& new_symlink, error_code& ec) noexcept;

Remarks: Achieves its postconditions as if by POSIX symlink().

27.10.?1.3 Create hard link [default_filesystem.create_hard_link]

virtual void create_hard_link(const path& to, const path& new_hard_link,
error_code& ec) noexcept;

Remarks: Achieves its postconditions as if by POSIX `link()`.

27.10.?.1.4 Create symlink [default filesystem.create_symlink]

virtual void create_symlink(const path& to, const path& new_symlink,
error_code& ec) noexcept;

Remarks: Achieves its postconditions as if by POSIX `symlink()`.

27.10.?.1.5 Current path [default filesystem.current_path]

virtual path current_path(error_code& ec);

Remarks: Obtains the current working directory as if by POSIX `getcwd()`.

virtual void current_path(const path& p, error_code& ec) noexcept;

Remarks: Achieves its postconditions as if by POSIX `chdir()`.

27.10.?.1.6 Equivalent [default filesystem.equivalent]

virtual bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;

Remarks: Determines equivalence of the two files as if by the values of the POSIX `stat` structure, obtained as if by `stat()` for the two paths, having equal `st_dev` values and equal `st_ino` values.

27.10.?.1.7 File size [default filesystem.file_size]

virtual uintmax_t file_size(const path& p, error_code& ec) noexcept;

Remarks: Determines the size as if by the value of the POSIX `stat` structure member `st_size` obtained as if by POSIX `stat()`.

27.10.?.1.8 Last write time [default filesystem.last_write_time]

virtual file_time_type last_write_time(const path& p, error_code& ec) noexcept;

Remarks: Determines the modification time as if by the value of the POSIX `stat` structure member `st_mtime` obtained as if by POSIX `stat()`.

virtual void last_write_time(const path& p, file_time_type new_time,
error_code& ec) noexcept;

Remarks: Sets the modification time as if by POSIX `futimens()`.

27.10.?.1.9 Permissions [default filesystem.permissions]

virtual void permissions(const path& p, perms prms, error_code& ec) noexcept;

Remarks: Applies the effective permission bits as if by POSIX `fchmodat()`.

27.10.?.1.10 Remove [default filesystem.remove]

virtual bool remove(const path& p, error_code& ec) noexcept;

Remarks: Removes the file as if by POSIX `remove()`.

27.10.?.1.11 Rename [default filesystem.rename]

virtual void rename(const path& old_p, const path& new_p, error_code& ec) noexcept;

Remarks: Renames the file as if by POSIX `rename()`.

27.10.?.1.12 Resize file [default filesystem.resize_file]

virtual void resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept;

Remarks: Achieves its postconditions as if by POSIX `truncate()`.

27.10.?.1.13 Space [default filesystem.space]

virtual space_info space(const path& p, error_code& ec) noexcept;

Remarks: The value of the `space_info` object is determined as if by using POSIX `statvfs` to obtain a POSIX

struct statvfs, and then multiplying its f_blocks, f_bfree, and f_bavail members by its f_frsize member, and assigning the results to the capacity, free, and available members respectively.

27.10.?1.14 Status [default_filesystem.status]

virtual file_status status(const path& p, error_code& ec) noexcept;

Remarks: Determines the attributes as if by POSIX stat().

27.10.?1.15 Symlink status [default_filesystem.symlink_status]

virtual file_status symlink_status(const path& p, error_code& ec) noexcept;

Remarks: Determines the attributes as if by POSIX lstat().

27.10.? Class directory_traverser [class.directory_traverser]

```
class directory_traverser {  
public:  
    directory_traverser() = default;  
    virtual ~directory_traverser() = default;  
  
    virtual const directory_entry& dereference() const = 0;  
    virtual void increment(error_code& ec) noexcept = 0;  
    virtual bool equal(const directory_traverser& rhs) const = 0;  
    virtual bool is_end() const noexcept = 0;  
    virtual unique_ptr<directory_traverser> clone() const = 0;  
};
```

The directory_traverser class defines the base class for the types of objects that represent how a directory in a particular filesystem implementation is traversed. An instance of such a type is either a past-the-end value, or points to an element of a sequence of directory_entry objects representing the contents of the directory.

Classes derived from directory_traverser shall satisfy all of the requirements of directory_traverser.

[Note: If a file is removed from or added to a directory after the construction of a directory_traverser for the directory, it is unspecified whether or not subsequently incrementing the directory_traverser will ever result in it pointing to the removed or added directory entry. See POSIX readdir_r. — end note]

27.10.?1 directory_traverser members [directory_traverser.members]

virtual const directory_entry& dereference() const = 0;

Precondition: !is_end()

Returns: The current element of the sequence being traversed.

virtual void increment(error_code& ec) = 0;

Precondition: !is_end()

Effects: Advances *this to the next element of the sequence being traversed. If there is no such element, makes *this a past-the-end value.

virtual bool equal(const directory_traverser& rhs) const = 0;

Precondition: rhs has the same dynamic type as *this.

Returns: true if rhs points to the same sequence element as *this, or if they are both past-the-end values, and false otherwise.

virtual bool is_end() const = 0;

Returns: true if *this represents a past-the-end value, and false otherwise.

virtual unique_ptr<directory_traverser> clone() const = 0;

Returns: a value v such that v->equal(*this) (and the dynamic type of *v is the same as the dynamic type of *this).

Revise the directory_iterator class synopsis as follows:

```
namespace std::filesystem {  
    class directory_iterator {
```

```

public:
    typedef directory_entry      value_type;
    typedef ptrdiff_t           difference_type;
    typedef const directory_entry* pointer;
    typedef const directory_entry& reference;
    typedef input_iterator_tag   iterator_category;

    // member functions
    directory_iterator() noexcept;
    explicit directory_iterator(const path& p);
    directory_iterator(const path& p, directory_options options);
    directory_iterator(const path& p, error_code& ec) noexcept;
    directory_iterator(const path& p, directory_options options,
                      error_code& ec) noexcept;
    directory_iterator(const directory_iterator& rhs);
    directory_iterator(directory_iterator&& rhs) noexcept;
    ~directory_iterator();

    directory_iterator& operator=(const directory_iterator& rhs);
    directory_iterator& operator=(directory_iterator&& rhs) noexcept;

    const directory_entry& operator*() const;
    const directory_entry* operator->() const;
    directory_iterator& operator++();
    directory_iterator& increment(error_code& ec) noexcept;

    // other members as required by 24.2.3, input iterators

private:
    unique_ptr<directory_traverser> traverser; // exposition only
};
}

```

Revise [directory_iterator.members] as follows:

27.10.13.1 directory_iterator members [directory_iterator.members]

```
directory_iterator() noexcept;
```

Effects: ~~Constructs the end iterator.~~ Initializes traverser with nullptr.

```

explicit directory_iterator(const path& p);
directory_iterator(const path& p, directory_options options);
directory_iterator(const path& p, error_code& ec) noexcept;
directory_iterator(const path& p, directory_options options, error_code& ec) noexcept;

```

Effects: ~~For the directory that p resolves to, constructs an iterator for the first element in a sequence of directory_entry elements representing the files in the directory, if any; otherwise the end iterator. However, if~~

~~(options & directory_options::skip_permissions_denied) != directory_options::none~~

~~and construction encounters an error indicating that permission to access p is denied, constructs the end iterator and does not report an error.~~ Initializes traverser with the result of current filesystem.load()->directory_traverser(p, options, ec) (with options equal to directory_options::none if not specified by the caller). The signatures with argument ec construct the end iterator if an error occurs.

Throws: As specified in Error reporting (27.5.6.5).

[Note: To iterate over the current directory, use directory_iterator(".") rather than directory_iterator(""). — end note]

```
directory_iterator(const directory_iterator& rhs);
```

Effects: Initializes traverser with rhs.traverser->clone().

```
directory_iterator(directory_iterator&& rhs);
```

Effects: ~~Constructs an object of class directory_iterator.~~ Initializes traverser with std::move(rhs.traverser).

Postconditions: ~~*this has the original value of rhs.~~

```
directory_iterator& operator=(const directory_iterator& rhs);
```

Effects: If *this and rhs are the same object, no effect. Otherwise, assigns rhs.traverser->clone() to traverser.

Returns: *this.

```
directory_iterator& operator=(directory_iterator&& rhs) noexcept;

Effects: If *this and rhs are the same object, the member has no effect. Assigns std::move(rhs.traverser) to traverser.

Postconditions: *this has the original value of rhs.

Returns: *this.

const directory_entry& operator*() const;

Effects: Equivalent to return traverser->dereference();.

const directory_entry* operator->() const;

Effects: Equivalent to return &traverser->dereference();.

directory_iterator& operator++();
directory_iterator& increment(error_code& ec) noexcept;

Effects: As specified by Input iterators (24.2.3). Invokes traverser->increment().

Returns: *this.

Throws: As specified in Error reporting (27.5.6.5).
```

Add the following to the end of [directory_iterator.nonmembers]:

```
bool operator==(const directory_iterator& lhs, const directory_iterator& rhs);
```

Returns: A value determined by Table ?.

Table ? — `operator==(const directory_iterator&, const directory_iterator&)` return value

	<code>rhs.traverser == nullptr rhs.traverser->is_end()</code>	<code>!(rhs.traverser == nullptr rhs.traverser->is_end())</code>
<code>lhs.traverser == nullptr lhs.traverser->is_end()</code>	true	false
<code>!(lhs.traverser == nullptr lhs.traverser->is_end())</code>	false	<code>lhs.traverser-> equal(*rhs.traverser)</code>

```
bool operator!=(const directory_iterator& lhs, const directory_iterator& rhs);
```

Effects: Equivalent to `!(lhs == rhs)`.

Insert a new section above [fs.op.funcs]:

27.10.? Filesystem injection [fs.inject]

27.10.?.1 Current filesystem [fs.inject.current]

```
extern atomic<filesystem*> current_filesystem;
```

The `current_filesystem` variable is for exposition only. An implementation is permitted to provide equivalent functionality without providing a variable with this name.

`current_filesystem` will be initialized to `&default_filesystem()` before the body of `main()` begins execution. [Note Implementations are encouraged to initialize before `main()` begins execution. --end note]

27.10.?.2 Inject filesystem [fs.inject.inject]

```
void inject_filesystem(filesystem& fs);
```

Effects: Equivalent to `current_filesystem = addressof(fs);`.

Remarks: `fs` must remain live until the next call to `inject_filesystem`, and until all pending calls to filesystem operation functions have completed execution.

27.10.?.3 Default filesystem [fs.inject.default]

```
filesystem& default_filesystem();
```


Returns: An object of type `default_filesystem_impl`. All invocations of this function will return the same object, and this object will not be destroyed during program execution.

Revise `[fs.op.copy]` paragraph 6 as follows:

Otherwise if `is_regular_file(f)`, then:

- If `(options & copy_options::directories_only) != copy_options::none`, then return.
- Otherwise if `(options & copy_options::create_symlinks) != copy_options::none`, then ~~create a symbolic link to the source file~~`create_symlink(from, to)`.
- Otherwise if `(options & copy_options::create_hard_links) != copy_options::none`, then ~~create a hard link to the source file~~`create_hard_link(from, to)`.
- Otherwise if `is_directory(t)`, then `copy_file(from, to/from.filename(), options)`.
- Otherwise, `copy_file(from, to, options)`.

Revise `[fs.op.copy_file]` as follows:

```
bool copy_file(const path& from, const path& to);  
bool copy_file(const path& from, const path& to, error_code& ec) noexcept;
```

Returns: `copy_file(from, to, copy_options::none)` OR `copy_file(from, to, copy_options::none, ec)`, respectively.

Throws: As specified in Error reporting (27.5.6.5).

```
bool copy_file(const path& from, const path& to, copy_options options);  
bool copy_file(const path& from, const path& to, copy_options options,  
               error_code& ec) noexcept;
```

~~*Requires:* At most one constant from each `copy_options` group is present in `options`:~~

~~*Effects:* Report a file already exists error as specified in Error reporting (27.5.6.5) if:~~

- ~~`exists(to)` and `equivalent(from, to)`, OR~~
- ~~`exists(to)` and `(options & (copy_options::skip_existing | copy_options::overwrite_existing | copy_options::update_existing)) == copy_options::none`:~~

~~Otherwise copy the contents and attributes of the file `from` resolves to to the file `to` resolves to if:~~

- ~~`!exists(to)`, OR~~
- ~~`exists(to)` and `(options & copy_options::overwrite_existing) != copy_options::none`, OR~~
- ~~`exists(to)` and `(options & copy_options::update_existing) != copy_options::none`, and `from` is more recent than `to`, determined as if by use of the `last_write_time` function.~~

~~Otherwise no effects.~~Invokes `current_filesystem.load()->copy_file(from, to, options, ec)` and handles any errors as specified in Error reporting (27.10.7).

~~*Returns:* `true` if the `from` file was copied, otherwise `false`. The signature with argument `ec` return `false` if an error occurs.~~The return value of the `copy_file()` member call.

Throws: As specified in Error reporting (27.5.6.5).

~~*Complexity:* At most one direct or indirect invocation of `status(to)`.~~

Revise `[fs.op.create_directory]` as follows:

```
bool create_directory(const path& p);  
bool create_directory(const path& p, error_code& ec) noexcept;
```

~~*Effects:* Establishes the postcondition by attempting to create the directory `p` resolves to, as if by POSIX `mkdir()` with a second argument of `static_cast<int>(perms::all)`. Creation failure because `p` resolves to an existing directory shall not be treated as an error.~~Invokes `current_filesystem.load()->create_directory(p, ec)` and handles any errors as specified in Error reporting (27.10.7).

~~*Postcondition:* `is_directory(p)`:~~

~~*Returns:* `true` if a new directory was created, otherwise `false`. The signature with argument `ec` returns `false` if an error occurs.~~The return value of the `create_directory()` member call.

Throws: As specified in Error reporting (27.5.6.5).

```
bool create_directory(const path& p, const path& existing_p);  
bool create_directory(const path& p, const path& existing_p, error_code& ec) noexcept;
```

~~*Effects:* Establishes the postcondition by attempting to create the directory `p` resolves to, with attributes~~

copied from directory `existing_p`. The set of attributes copied is operating system dependent. Creation failure because `p` resolves to an existing directory shall not be treated as an error. [Note: For POSIX based operating systems the attributes are those copied by native API `stat(existing_p.c_str(), &attributes_stat)` followed by `mkdir(p.c_str(), attributes_stat.st_mode)`. For Windows based operating systems the attributes are those copied by native API `CreateDirectoryExW(existing_p.c_str(), p.c_str(), 0)`. — end note] Invokes `current_filesystem.load()->create_directory(p, existing_p, ec)`; and handles any errors as specified in Error reporting (27.10.7).

Postcondition: `is_directory(p)`.

Returns: `true` if a new directory was created, otherwise `false`. The signature with argument `ec` returns `false` if an error occurs. The return value of the `create_directory()` member call.

Throws: As specified in Error reporting (27.5.6.5).

Revise `[fs.op.create_dir_symlink]` as follows:

```
void create_directory_symlink(const path& to, const path& new_symlink);
void create_directory_symlink(const path& to, const path& new_symlink,
                             error_code& ec) noexcept;
```

Effects: Establishes the postcondition, as if by `POSIX_symlink()` Invokes `current_filesystem.load()->create_directory_symlink(to, new_symlink, ec)` and handles any errors as specified in Error reporting (27.10.7).

Postcondition: `new_symlink` resolves to a symbolic link file that contains an unspecified representation of `to`.

Throws: As specified in Error reporting (27.5.6.5).

[Note: Some operating systems require symlink creation to identify that the link is to a directory. Portable code should use `create_directory_symlink()` to create directory symlinks rather than `create_symlink()` — end note]

[Note: Some operating systems do not support symbolic links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support symbolic links regardless of the operating system. — end note]

Revise `[fs.op.create_hard_lk]` as follows:

```
void create_hard_link(const path& to, const path& new_hard_link);
void create_hard_link(const path& to, const path& new_hard_link,
                     error_code& ec) noexcept;
```

Effects: Establishes the postcondition, as if by `POSIX_link()` Invokes `current_filesystem.load()->create_hard_link(to, new_hard_link, ec)` and handles any errors as specified in Error reporting (27.10.7).

Postcondition:

- `exists(to) && exists(new_hard_link) && equivalent(to, new_hard_link)`
- The contents of the file or directory `to` resolves to are unchanged.

Throws: As specified in Error reporting (27.5.6.5).

[Note: Some operating systems do not support hard links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support hard links regardless of the operating system. Some file systems limit the number of links per file. — end note]

Revise `[fs.op.create_symlink]` as follows:

```
void create_symlink(const path& to, const path& new_symlink);
void create_symlink(const path& to, const path& new_symlink,
                   error_code& ec) noexcept;
```

Effects: Establishes the postcondition, as if by `POSIX_symlink()` Invokes `current_filesystem.load()->create_symlink(to, new_symlink, ec)` and handles any errors as specified in Error reporting (27.10.7).

Postcondition: `new_symlink` resolves to a symbolic link file that contains an unspecified representation of `to`.

Throws: As specified in Error reporting (27.5.6.5).

[Note: Some operating systems do not support symbolic links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support symbolic links regardless of the operating system. — end note]

Revise [fs.op.current_path] as follows:

```
path current_path();  
path current_path(error_code& ec);
```

Effects: Invokes `current_filesystem.load()`->`current_path(ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: The absolute path of the current working directory, obtained as if by POSIX `getcwd()`. The signature with argument `ec` returns `path()` if an error occurs. The return value of the `current_path()` member call.

Throws: As specified in Error reporting (27.5.6.5).

Remarks: The current working directory is the directory, associated with the process, that is used as the starting location in pathname resolution for relative paths.

[Note: The `current_path()` name was chosen to emphasize that the return is a path, not just a single directory name.

The current path as returned by many operating systems is a dangerous global variable. It may be changed unexpectedly by a third-party or system library functions, or by another thread. — *end note*]

```
void current_path(const path& p);  
void current_path(const path& p, error_code& ec) noexcept;
```

Effects: Establishes the postcondition, as if by POSIX `chdir()` Invokes `current_filesystem.load()`->`current_path(p, ec)` and handles any errors as specified in Error reporting (27.10.7).

Postcondition: `equivalent(p, current_path())`.

Throws: As specified in Error reporting (27.5.6.5).

[Note: The current path for many operating systems is a dangerous global state. It may be changed unexpectedly by a third-party or system library functions, or by another thread. — *end note*]

Revise [fs.op.equivalent] as follows:

```
bool equivalent(const path& p1, const path& p2);  
bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;
```

Effects: ~~Determines file status `s1` and `s2`, as if by `status(p1)` and `status(p2)`, respectively~~ Invokes `current_filesystem.load()`->`equivalent(p1, p2, ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: ~~`true`, if `s1 == s2` and `p1` and `p2` resolve to the same file system entity, else `false`. The signature with argument `ec` returns `false` if an error occurs. The return value of the `equivalent()` member call.~~

Two paths are considered to resolve to the same file system entity if two candidate entities reside on the same device at the same location. This is determined as if by the values of the POSIX `stat` structure, obtained as if by `stat()` for the two paths, having equal `st_dev` values and equal `st_ino` values.

Throws: ~~`filesystem_error` if `(!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2))`, otherwise `as`~~ specified in Error reporting (27.5.6.5).

Revise [fs.op.file_size] as follows:

```
uintmax_t file_size(const path& p);  
uintmax_t file_size(const path& p, error_code& ec) noexcept;
```

Effects: Invokes `current_filesystem.load()`->`file_size(p, ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: If `!exists(p) || !is_regular_file(p)` an error is reported 27.5.6.5. Otherwise, the size in bytes of the file `p` resolves to, determined as if by the value of the POSIX `stat` structure member `st_size` obtained as if by POSIX `stat()`. The signature with argument `ec` returns `static_cast<uintmax_t>(-1)` if an error occurs. The return value of the `file_size()` member call.

Throws: As specified in Error reporting (27.5.6.5).

Revise [fs.op.hard_lk_ct] as follows:

```
uintmax_t hard_link_count(const path& p);  
uintmax_t hard_link_count(const path& p, error_code& ec) noexcept;
```

Effects: Invokes `current_filesystem.load()`->`hard link count(p, ec)` and handles any errors as specified in

Error reporting (27.10.7).

Returns: ~~The number of hard links for p.~~ The signature with argument `ec` returns `static_cast<uintmax_t>(-1)` if an error occurs.The return value of the `hard_link_count()` member call.

Throws: As specified in Error reporting (27.5.6.5).

Revise `[fs.op.last_write_time]` as follows:

```
file_time_type last_write_time(const path& p);
file_time_type last_write_time(const path& p, error_code& ec) noexcept;
```

Effects: Invokes `current_filesystem.load()->last_write_time(p, ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: ~~The time of last data modification of p, determined as if by the value of the POSIX `stat` structure member `st_mtime` obtained as if by `POSIX::stat()`.~~ The signature with argument `ec` returns `file_time_type::min()` if an error occurs.The return value of the `last_write_time()` member call.

Throws: As specified in Error reporting (27.5.6.5).

```
void last_write_time(const path& p, file_time_type new_time);
void last_write_time(const path& p, file_time_type new_time,
    error_code& ec) noexcept;
```

Effects: Sets the time of last data modification of the file resolved to by p to `new_time`, as if by `POSIX::futimens()` Invokes `current_filesystem.load()->last_write_time(p, new_time, ec)` and handles any errors as specified in Error reporting (27.10.7).

Throws: As specified in Error reporting (27.5.6.5).

[Note: A postcondition of `last_write_time(p) == new_time` is not specified since it might not hold for file systems with coarse time granularity. — end note]

Revise `[fs.op.permissions]` as follows:

```
void permissions(const path& p, perms prms);
void permissions(const path& p, perms prms, error_code& ec) noexcept;
```

Requires: `!((prms & perms::add_perms) != perms::none <br> && (prms & perms::remove_perms) != perms::none).`

Effects: ~~Applies the effective permissions bits from `prms` to the file p resolves to, as if by `POSIX::fchmodat()`.~~ The effective permission bits are determined as specified in Table 150.

Table 150 — Effects of permission bits

Bits present in <code>prms</code>	Effective bits applied
Neither <code>add_perms</code> nor <code>remove_perms</code>	<code>prms & perms::mask</code>
<code>add_perms</code>	<code>status(p).permissions() (prms & perms::mask)</code>
<code>remove_perms</code>	<code>status(p).permissions() & (prms & perms::mask)</code>

Invokes `current_filesystem.load()->permissions(p, prms, ec)` and handles any errors as specified in Error reporting (27.10.7).

[Note: Conceptually permissions are viewed as bits, but the actual implementation may use some other mechanism. — end note]

Throws: As specified in Error reporting (27.5.6.5).

Revise `[fs.op.read_symlink]` as follows:

```
path read_symlink(const path& p);
path read_symlink(const path& p, error_code& ec);
```

Effects: Invokes `current_filesystem.load()->read_symlink(p, ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: ~~If p resolves to a symbolic link, a path object containing the contents of that symbolic link. The signature with argument `ec` returns `path()` if an error occurs.~~ The return value of the `read_symlink()` member call.

Throws: As specified in Error reporting (27.5.6.5). [Note: It is an error if p does not resolve to a symbolic link. — end note]

Revise `[fs.op.remove]` as follows:

```
bool remove(const path& p);
bool remove(const path& p, error_code& ec) noexcept;
```

Effects: ~~If exists(symlink_status(p,ec)), it is removed as if by POSIX remove()~~ Invokes current_filesystem.load()->remove(p, ec) and handles any errors as specified in Error reporting (27.10.7).

[Note: A symbolic link is itself removed, rather than the file it resolves to being removed. **—end note]**

Postcondition: ~~!exists(symlink_status(p));~~

Returns: ~~false if p did not exist, otherwise true. The signature with argument ec returns false if an error occurs.~~ The return value of the remove() member call

Throws: As specified in Error reporting (27.5.6.5).

Revise [fs.op.rename] as follows:

```
void rename(const path& old_p, const path& new_p);
void rename(const path& old_p, const path& new_p, error_code& ec) noexcept;
```

Effects: ~~Renames old_p to new_p, as if by POSIX rename()~~ Invokes current_filesystem.load()->rename(old_p, new_p, ec) and handles any errors as specified in Error reporting (27.10.7).

[Note: If old_p and new_p resolve to the same existing file, no action is taken. Otherwise, if new_p resolves to an existing non-directory file, it is removed, while if new_p resolves to an existing directory, it is removed if empty on POSIX-compliant operating systems but is an error on some other operating systems. A symbolic link is itself renamed, rather than the file it resolves to being renamed. **—end note]**

Throws: As specified in Error reporting (27.5.6.5).

Revise [fs.op.resize_file] as follows:

```
void resize_file(const path& p, uintmax_t new_size);
void resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept;
```

Effects: Invokes current_filesystem.load()->resize_file(p, new_size, ec) and handles any errors as specified in Error reporting (27.10.7).

Postcondition: ~~file_size() == new_size;~~

Throws: As specified in Error reporting (27.5.6.5).

Remarks: Achieves its postconditions as if by POSIX truncate().

Revise [fs.op.space] as follows:

```
space_info space(const path& p);
space_info space(const path& p, error_code& ec) noexcept;
```

Effects: Invokes current_filesystem.load()->space(p, ec) and handles any errors as specified in Error reporting (27.10.7).

Returns: ~~An object of type space_info. The value of the space_info object is determined as if by using POSIX statvfs to obtain a POSIX struct statvfs, and then multiplying its f_blocks, f_bfree, and f_bavail members by its f_frsize member, and assigning the results to the capacity, free, and available members respectively. Any members for which the value cannot be determined shall be set to static_cast<uintmax_t>(-1). For the signature with argument ec, all members are set to static_cast<uintmax_t>(-1) if an error occurs.~~ The return value of the space() member call.

Throws: As specified in Error reporting (27.5.6.5).

Remarks: ~~The value of member space_info::available is operating system dependent. [Note: available may be less than free. —end note]~~

Revise [fs.op.status] as follows:

```
file_status status(const path& p);
```

Effects: As if:

```
error_code ec;
file_status result = status(p, ec);
if (result == file_type::none)
    throw filesystem_error(implementation-supplied-message , p, ec);
```

```
return result;
```

Returns: See above.

Throws: `filesystem_error` [*Note:* result values of `file_status(file_type::not_found)` and `file_status(file_type::unknown)` are not considered failures and do not cause an exception to be thrown. — *end note*]

```
file_status status(const path& p, error_code& ec) noexcept;
```

Effects: If possible, determines the attributes of the file `p` resolves to, as if by `POSIX_stat()`. If, during attribute determination, the underlying file system API reports an error, sets `ec` to indicate the specific error reported. Otherwise, `ec.clear()`. Equivalent to return `current_filesystem.load()->status(p, ec)`.

[*Note:* This allows users to inspect the specifics of underlying API errors even when the value returned by `status()` is not `file_status(file_type::none)`. — *end note*]

Returns: If `ec != error_code()`:

- If the specific error indicates that `p` cannot be resolved because some element of the path does not exist, return `file_status(file_type::not_found)`;
- Otherwise, if the specific error indicates that `p` can be resolved but the attributes cannot be determined, return `file_status(file_type::unknown)`;
- Otherwise, return `file_status(file_type::none)`;

[*Note:* These semantics distinguish between `p` being known not to exist, `p` existing but not being able to determine its attributes, and there being an error that prevents even knowing if `p` exists. These distinctions are important to some use cases. — *end note*]

Otherwise,

- If the attributes indicate a regular file, as if by `POSIX_S_ISREG`, return `file_status(file_type::regular)`; [*Note:* `file_type::regular` implies appropriate `<fstream>` operations would succeed, assuming no hardware, permission, access, or file system race errors. Lack of `file_type::regular` does not necessarily imply `<fstream>` operations would fail on a directory. — *end note*]
- Otherwise, if the attributes indicate a directory, as if by `POSIX_S_ISDIR`, return `file_status(file_type::directory)`. [*Note:* `file_type::directory` implies `directory_iterator(p)` would succeed. — *end note*]
- Otherwise, if the attributes indicate a block special file, as if by `POSIX_S_ISBLK`, return `file_status(file_type::block)`;
- Otherwise, if the attributes indicate a character special file, as if by `POSIX_S_ISCHR`, return `file_status(file_type::character)`;
- Otherwise, if the attributes indicate a fifo or pipe file, as if by `POSIX_S_ISFIFO`, return `file_status(file_type::fifo)`;
- Otherwise, if the attributes indicate a socket, as if by `POSIX_S_ISSOCK`, return `file_status(file_type::socket)`;
- Otherwise, return `file_status(file_type::unknown)`;

Remarks: If a symbolic link is encountered during pathname resolution, pathname resolution continues using the contents of the symbolic link.

Revise `[fs.op.symlink_status]` as follows:

```
file_status symlink_status(const path& p);
```

Effects: As if:

```
error_code ec;  
file_status result = status_symlink(p, ec);  
if (result == file_type::none)  
    throw filesystem_error(implementation-supplied-message, p, ec);  
return result;
```

Returns: See above.

Throws: `filesystem_error` [*Note:* result values of `file_status(file_type::not_found)` and `file_status(file_type::unknown)` are not considered failures and do not cause an exception to be thrown. — *end note*]

```
file_status symlink_status(const path& p, error_code& ec) noexcept;
```

Effects: Same as `status()`, above, except that the attributes of `p` are determined as if by `POSIX_lstat()`.
Equivalent to `return current_filesystem.load()->symlink_status(p, ec)`.

Returns: Same as `status()`, above, except that if the attributes indicate a symbolic link, as if by `POSIX_S_ISLNK`,
`return file_status(file_type::symlink)`. The signature with argument `ec` returns
`file_status(file_type::none)` if an error occurs.

Remarks: Pathname resolution terminates if `p` names a symbolic link.

Throws: As specified in Error reporting (27.5.6.5).

Revise `[fs.op.system_complete]` as follows:

```
path system_complete(const path& p);  
path system_complete(const path& p, error_code& ec);
```

Effects: Composes an absolute path from `p`, using the same rules used by the operating system to resolve a path passed as the filename argument to standard library open functions. Invokes `current_filesystem.load()->system_complete(p, ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: The composed path. The signature with argument `ec` returns `path()` if an error occurs. The return value of the `system_complete()` member call.

Postcondition: For the returned path, `rp.is_absolute()` is true.

Throws: As specified in Error reporting (27.5.6.5).

[*Example:* For POSIX based operating systems, `system_complete(p)` has the same semantics as `absolute(p, current_path())`:

For Windows based operating systems, `system_complete(p)` has the same semantics as `absolute(p, current_path())` if `p.is_absolute()` || `!p.has_root_name()` or `p` and `base` have the same `root_name()`. Otherwise it acts like `absolute(p, cwd)` is the current directory for the `p.root_name()` drive. This will be the current directory for that drive the last time it was set, and thus may be residue left over from a prior program run by the command processor. Although these semantics are useful, they may be surprising. — end example]

Revise `[fs.op.temp_dir_path]` as follows:

```
path temp_directory_path();  
path temp_directory_path(error_code& ec);
```

Effects: Invokes `current_filesystem.load()->temp_directory_path(ec)` and handles any errors as specified in Error reporting (27.10.7).

Returns: An unspecified directory path suitable for temporary files. An error shall be reported if `!exists(p)` || `!is_directory(p)`, where `p` is the path to be returned. The signature with argument `ec` returns `path()` if an error occurs. The return value of the `temp_directory_path()` member call.

Throws: As specified in Error reporting (27.5.6.5).

[*Example:* For POSIX based operating systems, an implementation might return the path supplied by the first environment variable found in the list `TMPDIR`, `TMP`, `TEMP`, `TEMPDIR`, or if none of these are found, `"/tmp"`.

For Windows based operating systems, an implementation might return the path reported by the Windows `GetTempPath` API function. — end note]